AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY

**Amrita School of AI**
COIMBATORE CAMPUS

23AID304 · HIGH-PERFORMANCE AND CLOUD COMPUTING

# Course Handbook

Everything about how this course runs. Read it in week one and it answers most of the questions you were going to ask.

Prepared by **Dr. Abhijith Anandakrishnan**
Assistant Professor, Amrita School of AI

COURSE
23AID304 High-Performance and Cloud Computing

STRUCTURE
L-T-P-C 2-0-2-3

CONTACT
2 lecture hours + one 2-hour lab, weekly

DURATION
15 teaching weeks

OFFERING
Odd Semester 2026-27

# What this course is about

Until about 2005 you made a program faster by waiting: clock speeds doubled every couple of years and yesterday's code ran twice as fast on today's machine. That stopped. Performance now comes from using **more** of a machine at once, and no compiler will do that for you.

This course teaches you to use a modern machine properly: all the cores of a processor, all the processors of a cluster, the GPUs attached to them, and the cloud when you need more than you own. It ends with the computation that currently consumes more of the world's hardware than any other, training and serving large models, and shows that it is the same problem again.

You will write real parallel code, measure it honestly, and be able to say in advance how fast it could possibly go.

# Learning outcomes

| | OUTCOME |
|---|---|
| **CO1** | Apply high-performance computing in different research fields |
| **CO2** | Design OpenMP/MPI programmes and CUDA programmes |
| **CO3** | Simulate on a cloud computing system |
| **CO4** | Evaluate how the convergence of HPC and AI is transforming data science |

# How the course is assessed

| COMPONENT | WEIGHT | INTERNAL OR EXTERNAL |
|---|---|---|
| **Continuous evaluation tests** (three, on Evolveus) | **30%** | Internal |
| **Lab and class assignments** | **10%** | Internal |
| **Course project** | **30%** | Internal |
| **End-semester examination** | **30%** | External |

**NOTE · THIS SUPERSEDES THE PATTERN PRINTED IN THE CURRICULUM DOCUMENT**

The 2021 curriculum booklet lists assignments 20, quizzes 20, mini project 20, mid-term 10, end-semester 30. That scheme is **no longer in force** for full-credit courses. The table above is what applies.

## Continuous evaluation, 30%

Three tests, each worth 10 percent, delivered on the **Evolveus** platform.

| TEST | COVERS | ROUGHLY WHEN |
|---|---|---|
| CE Test 1 | Units 0 and 1: architecture, memory, scaling metrics, roofline | Around week 5 |
| CE Test 2 | Unit 2: OpenMP and MPI | Around week 10 |
| CE Test 3 | Units 3 and 4: GPU and CUDA, cloud, HPC-AI convergence | Around week 14 |

There is no separate mid-term examination. These three tests replace it.

**NOTE · HOW TO PREPARE**

Each test draws on the unit it covers. The self-assessment questions at the end of every notes booklet are the closest available guide to their style and difficulty, and their full solutions are printed with them. Work those before each test rather than the night before.

Where a question is multiple choice, one or more options may be correct, a correct response earns the stated mark, and a wrong response loses a quarter of it. Guessing blind has an expected value of zero, so leave what you do not know.

## Lab and class assignments, 10%

Earned continuously across the semester:

| ITEM | SHARE OF THE COURSE | WHAT IT IS |
|---|---|---|
| Assignments | 6% | Four, one per block of the syllabus |
| Coding exercises | 2% | Auto-graded, submitted as a single file |
| Lab sheets | 2% | Nine lab reports |

**PITFALL · TEN PERCENT IS NOT A REASON TO SKIP THEM**

This is the smallest component and by far the largest share of the work. It is also where every skill the other 90 percent tests actually gets built: the CE tests ask you to reason about things you will only understand by having measured them, and the project is impossible without the toolchain and the method the labs establish. Students who treat this component as optional reliably do badly in the other three.

## Course project, 30%

The largest single component, assessed at two reviews.

| REVIEW | WEIGHT | WHEN | WHAT IS ASSESSED |
|---|---|---|---|
| First review | 10% | Around week 8 | Problem, serial baseline, roofline placement, parallel design, and a credible plan |
| Final review | 20% | Final week | Implementation, correctness evidence, scaling study, analysis, report and presentation |

The catalogue of project options, the deliverables and the full mark breakdown are in `mini-project/`. Read it in week 6, not week 8.

## End-semester examination, 30%

Three hours, 100 marks, no choices, covering all four units with additional emphasis on Units 3 and 4.

## How labs, exercises and assignments are submitted and graded

All three are delivered through **Classroom 50**, a GitHub Classroom replacement that the School runs on its own GitHub organisation. Every item is a repository created for you when you accept it; you commit your work there and submit with one command. Marks come from tests, and the report of every test appears on your repository within a couple of minutes.

The flow, once per item:

```
gh student accept Amrita-School-of-AI aim-a-hpc lab04-openmp-and-races
git clone <the URL it prints>
# work; make check runs the public tests
git add -A && git commit -m "what you changed"
gh student submit
```

(`aim-b-hpc` for the B section.) The grader's report is published as a GitHub Release on your repository, one line per check with the reason for any failure, and a green or red mark appears on the commit. Submit as often as you like before the deadline; the last submission counts.

What is tested is the part of each item a machine can check:

| KIND OF CHECK | EXAMPLES |
|---|---|
| Correctness | your function against a reference on inputs you have not seen, at awkward sizes and rank counts |
| Properties | the blocked version faster than the naive one, dynamic faster than static on an unbalanced loop |
| Analysis | your program computing speedup, efficiency, Karp-Flatt or the roofline exactly on hidden data |
| Committed measurements | the CSV you produced on the cluster, checked for shape and plausibility |
| Closed-form answers | cache-line arithmetic, ridge points, occupancy, predicted communication time in `answers.yaml` |

The grading machine is a two-core virtual machine without a GPU. Timing checks therefore ask only for modest, robust speedups, and correctness carries most of the marks. Items that need a GPU are compiled on the grading machine and their GPU tests run on `asaicompute` after the deadline; the report tells you which marks are pending. The four assignments also carry a short written section, `EXPLANATION.md`, of at most 250 words, graded against a published rubric.

Two files in every repository belong to the course and are restored when you submit: `.gitignore` and `.github/`. Do not edit them.

**PITFALL · WHY DETERMINISM IS CHECKED, AND CHECKED REPEATEDLY**

A parallel program with a data race often produces the right answer most of the time. Running the tests once would let it pass. The grader runs them several times at several thread counts precisely so that "usually correct" fails, which is the honest outcome.

**NOTE · A FAST WRONG ANSWER SCORES NOTHING**

Every timing check is skipped unless the correctness checks for the same code have passed. An empty function is infinitely fast; it earns no speedup marks.

## Policies

**Late submissions.** Marked out of 80 percent within 48 hours; not accepted after that except with documented cause. Tell me before the deadline, not after.

The documented-cause clause was applied to every Classroom 50 item on 2 November 2026. The School's GitHub Actions quota ran out on 22 October, so nothing submitted after 21:53 that evening was graded, and the VPN that reaches `asaicompute` is down. All 23 graded items therefore run to 7 November 2026, the 80 percent factor applies across that whole window instead of 48 hours, and work blocked by the grading outage counts as on time. `classroom-items.pdf` has the weighting table.

**Collaboration.** Discussing ideas, approaches and errors is encouraged and is how people actually learn. Submitting code you did not write is not. Every submission is compared against every other, and against the reference, using a similarity measure that ignores variable renaming and reformatting.

**AI tools.** You may use them to learn, to explain an error, or to explore an approach. You must be able to explain every line you submit. If you cannot, the work is not yours regardless of how it was produced. Vivas will be used where a submission and a student's understanding do not match.

**Attendance.** Institutional rules apply. The lab sessions in particular are not recoverable by reading: the cluster work has to be done on the cluster.

# Week-by-week plan

Each week has two lecture hours and one two-hour lab block. Weeks are numbered from the start of instruction so that this plan is reusable in any offering.

| WEEK | LECTURES | LAB | NOTES |
|---|---|---|---|
| 1 | Why parallel computing; the three walls | Environment setup, compiler, timing correctly | Unit 0 |
| 2 | Memory hierarchy, cache lines, locality | Measuring loop order and cache effects | Unit 0 |
| 3 | Processes and threads; Flynn; shared vs distributed | Cluster login, SLURM basics, first batch job | Unit 1 |
| 4 | Parallel algorithm design (PCAM); load balance | Decomposition on paper, then in code | Unit 1 |
| 5 | Speedup, efficiency, Amdahl, Gustafson, Karp-Flatt | Scaling study, strong and weak | Unit 1 · **CE Test 1** |
| 6 | Arithmetic intensity and the roofline model | Roofline placement of your own kernels | Unit 1 |
| 7 | OpenMP: regions, worksharing, data sharing | First OpenMP programs | Unit 2 |
| 8 | Reductions, races, false sharing, scheduling | Reproduce a race, then a false-sharing collapse | Unit 2 · **project first review** |
| 9 | Tasks; efficient OpenMP for matrix computation | Blocking and parallelising a matrix multiply | Unit 2 |
| 10 | MPI: ranks, point-to-point, the deadlock | First MPI programs, then break them deliberately | Unit 2 · **CE Test 2** |
| 11 | Collectives; derived datatypes; domain decomposition | Halo exchange stencil | Unit 2 |
| 12 | GPU architecture; SIMT; the CUDA model | First CUDA kernels on the cluster GPUs | Unit 3 |
| 13 | Memory hierarchy, coalescing, shared memory tiling | Tiled matrix multiply, transpose | Unit 3 |
| 14 | Reductions, streams, cuBLAS and cuDNN, profiling | Profile a kernel, place it on a measured roofline | Unit 3 · **CE Test 3** |
| 15 | Cloud models and economics; HPC-AI convergence | **Project final review** and presentations | Unit 4 |

**Assessment points.** Assignments are released in weeks 3, 6, 9 and 12. Continuous evaluation tests fall around weeks 5, 10 and 14. The course project is proposed in week 6, has its **first review around week 8**, and its **final review** in the last week.

# The compute you have

Almost all lab work runs on the School of AI's SLURM cluster.

| HOST | CPU | MEMORY | GPU |
|---|---|---|---|
| `asaicomputemaster` | 2 × AMD EPYC 9354, 64 cores | ~724 GiB | 2 × RTX 6000 Ada 48 GB |
| `asaicomputenode02` | AMD EPYC 9654, 96 cores | ~755 GiB | 2 × A100 80 GB |
| `asaicomputenode03` | AMD EPYC 9654, 96 cores | ~755 GiB | 2 × A100 80 GB |

Aggregate: 3 nodes, 256 CPUs, 6 GPUs, roughly 2.2 TB of memory. SLURM 24.11, CUDA 13.0, Ubuntu 24.04.

**ON ASAICOMPUTE · CLUSTER ETIQUETTE, WHICH IS ALSO EXAM-RELEVANT**

Never run compute on the login node. Ask for what you need and no more: requesting 96 cores for a serial job blocks your classmates and teaches you nothing. Always set a `--time` limit. Use `srun` for short interactive work and `sbatch` for anything real. Check `squeue` before complaining that the cluster is slow.

A minimal batch script:

```
#!/bin/bash
#SBATCH --job-name=lab04
#SBATCH --output=lab04-%j.out
#SBATCH --cpus-per-task=16
#SBATCH --mem=16G
#SBATCH --time=00:10:00

export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK
export OMP_PROC_BIND=close
export OMP_PLACES=cores

./myprogram
```

**PITFALL · THREAD PINNING IS NOT OPTIONAL IN THIS COURSE**

Without `OMP_PROC_BIND=close` the operating system may migrate your threads mid-run, which arrives with cold caches and, on a NUMA machine, possibly on the wrong side of your data. Unpinned timings are not reproducible and will not be accepted in scaling studies.

# What you need on your own machine

You can do most non-GPU work locally. Everything here is free and available on Linux, macOS and WSL2.

```
# Debian / Ubuntu
sudo apt install build-essential gdb git openmpi-bin libopenmpi-dev \
                 linux-tools-common python3-matplotlib
```

| TOOL | WHY |
|---|---|
| `gcc` 13+ | OpenMP 5.x support |
| Open MPI or MPICH | MPI exercises |
| `git` | Fetching material and exercises |
| `perf` | Cache-miss counters for the memory exercises |
| Python + matplotlib | Plotting scaling studies |

CUDA work needs a GPU, so it runs on the cluster. A Colab notebook is provided as a fallback for anyone who cannot reach the cluster during a submission window.

# How to do well in this course

**Measure before you optimise.** Every year, several students spend a week hand-tuning a loop that was already at 90 percent of its achievable ceiling. Ten minutes of arithmetic with the roofline model would have told them to stop.

**Fix the serial code first.** Parallelising a cache-hostile loop gives you many cores all waiting on memory. The matrix multiply worked example in Unit 2 gains 13.5x before a single thread is created.

**Report results honestly.** State the machine, compiler and flags, problem size, thread or rank count, number of repetitions, and which statistic you are quoting. A number without those is an anecdote.

**Read the error, then the code, then the assumption.** Most parallel bugs are an assumption about ordering that the hardware never promised.

**Ask early.** A ten-minute conversation in week 4 is worth more than a week of being stuck in week 12.

# Reading

The notes booklets for this course are self-contained: you are not required to buy or read any textbook. The following are there if you want to go further.

- Sterling, Anderson, Brodowicz, *High Performance Computing: Modern Systems and Practices*. The closest match to this syllabus.
- Pacheco, *An Introduction to Parallel Programming*. The gentlest introduction to OpenMP and MPI.
- Hennessy and Patterson, *Computer Architecture: A Quantitative Approach*. The reference for everything in Units 0 and 1.
- Sanders and Kandrot, *CUDA by Example*. Dated in its API, still excellent on the ideas.
- Williams, Waterman, Patterson, *Roofline: An Insightful Visual Performance Model* (CACM, 2009). Short, and worth reading in the original.
- The OpenMP and MPI specifications. Genuinely readable, and the only authority.

# Getting help

Bring me, in this order: what you expected, what happened, the exact error text, and the smallest program that still shows the problem. Producing that last item solves the problem about half the time on its own.